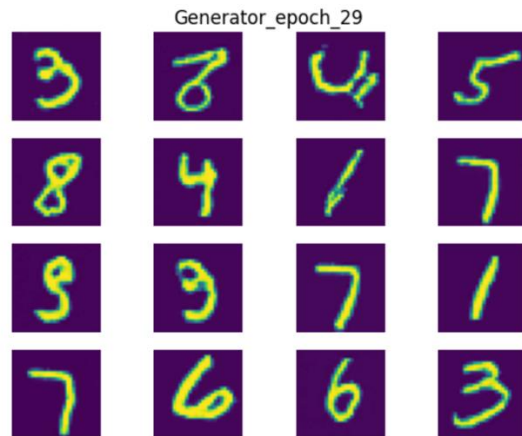


HW2: Generative Adversarial Network

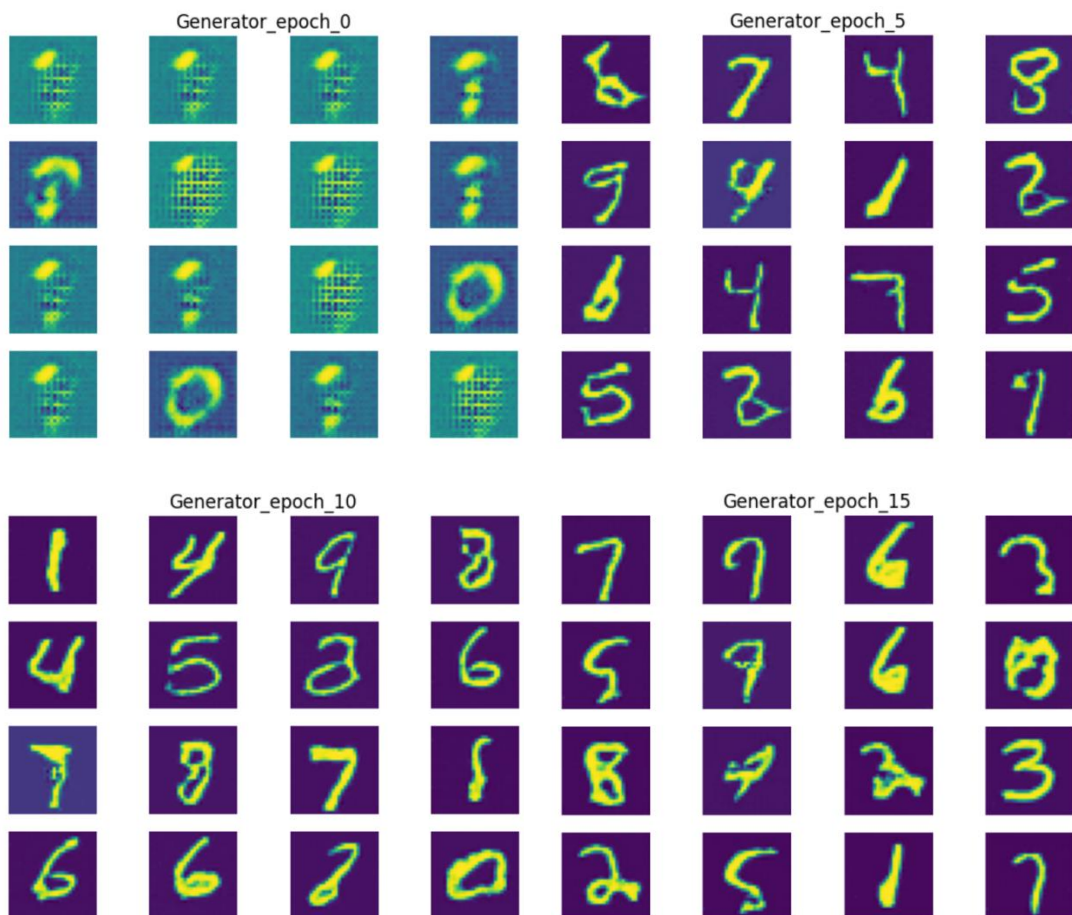
1. Show your generated images of LSGAN and DCGAN as described in “IV Implementation 3.”. Compare the result of LSGAN and DCGAN at least in two aspects.

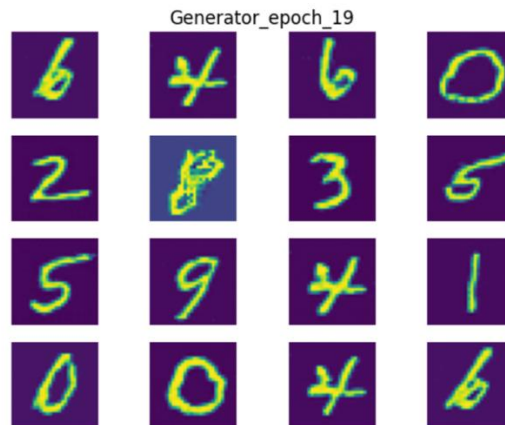
DCGAN Model Generated Images





LSGAN Model Generated Images





Analysis of Generated Digit Quality

I compared the results of the DCGAN and LSGAN models across three aspects: realism, sharpness, and diversity.

(1) Realism

DCGAN Model:

At the early epochs (e.g., Epochs 0 and 5), the digits appear mostly indistinguishable and noisy, resembling random blobs. As training progresses (e.g., Epochs 15-29), the digits improve in realism. By Epoch 29, the digits are clearer and resemble handwritten numbers; however, occasional artifacts or irregularities may still be observed (e.g., the digits "6" and "9" may exhibit inconsistent shapes). Overall, the realism is acceptable by Epochs 25 and 29, although some digits may still lack sharpness and clarity.

LSGAN Model:

At early epochs (e.g., Epoch 0), the results exhibit noise, although they are slightly less noisy than those produced by DCGAN, with some samples showing hints of structure. By Epoch 5, the digits become more distinguishable and realistic, with certain digits (such as "4" or "7") appearing clearer and sharper in comparison to DCGAN. At later epochs (e.g., Epoch 19), the digits are well-formed, smooth, and realistic, displaying fewer distortions or irregularities compared to DCGAN.

Summary:

LSGAN generates more realistic and well-formed digits, particularly during the early and middle epochs, whereas DCGAN requires a longer time to converge to realistic outputs.

(2) Sharpness

DCGAN Model:

Sharpness improves significantly over the epochs. By Epoch 29, most digits are sharp and recognizable; however, some samples may still exhibit blurred edges or inconsistencies in line thickness.

LSGAN Model:

Digits generated by LSGAN demonstrate improved sharpness earlier in the training process (e.g., Epochs 5-10), featuring clearer boundaries and smoother lines. By Epoch 19, the generated digits exhibit consistent sharpness with minimal artifacts or blurriness.

Summary:

LSGAN consistently achieves better sharpness across epochs compared to DCGAN.

(3) Diversity

DCGAN Model:

The diversity of generated digits improves gradually across epochs. By Epoch 29, the DCGAN produces a wide variety of digits (0-9); however, some patterns and digits are repeated, and certain representations may not fully resemble realistic numbers (e.g., distorted "9" or "6"). Additionally, there are irregularities in the diversity of representation, with some digits appearing more consistently than others.

LSGAN Model:

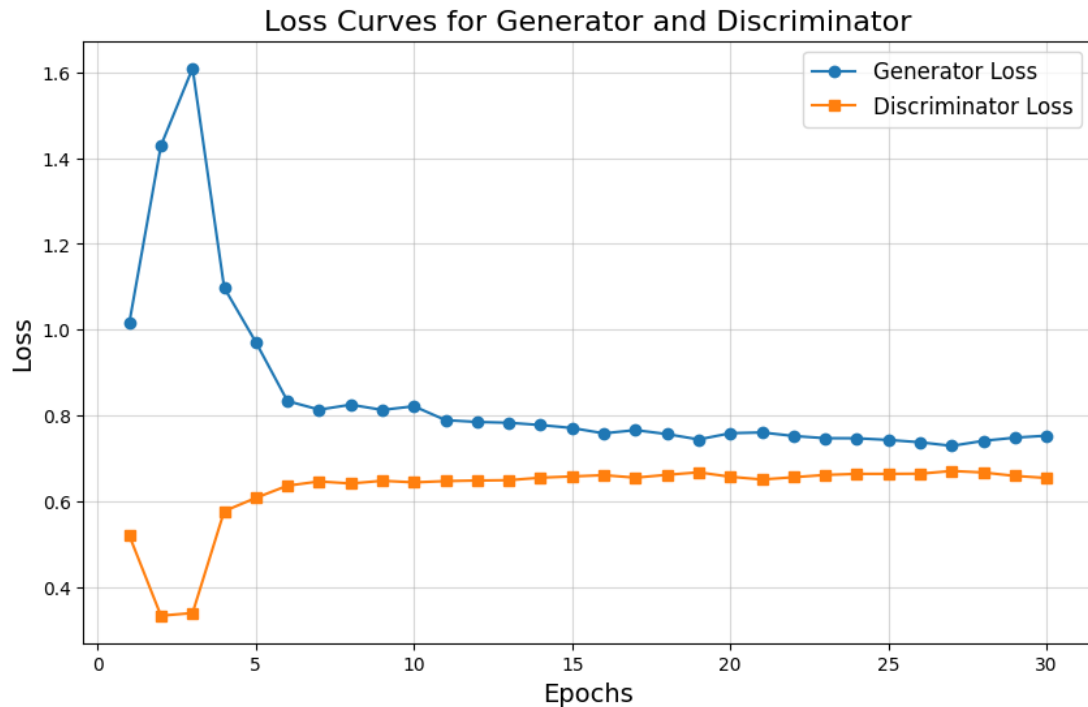
LSGAN begins generating diverse digits as early as Epoch 5, showcasing a wider range of digit styles with reduced distortion. The variability in writing styles, such as thicker or thinner digits, is more balanced, and by Epoch 19, it appears to produce distinct examples of all digits.

Summary:

LSGAN demonstrates greater diversity, producing fewer repetitive or distorted digits in its outputs.

2. Show your loss curves of DCGAN and LSGAN as mentioned in “IV Implementation 2.”. Try to describe tendency of the curves, and explain why the curves looks like that and how the loss value related to the quality of generated images.

Loss Curves of DCGAN Model



(1) Description of the DCGAN Loss Curves

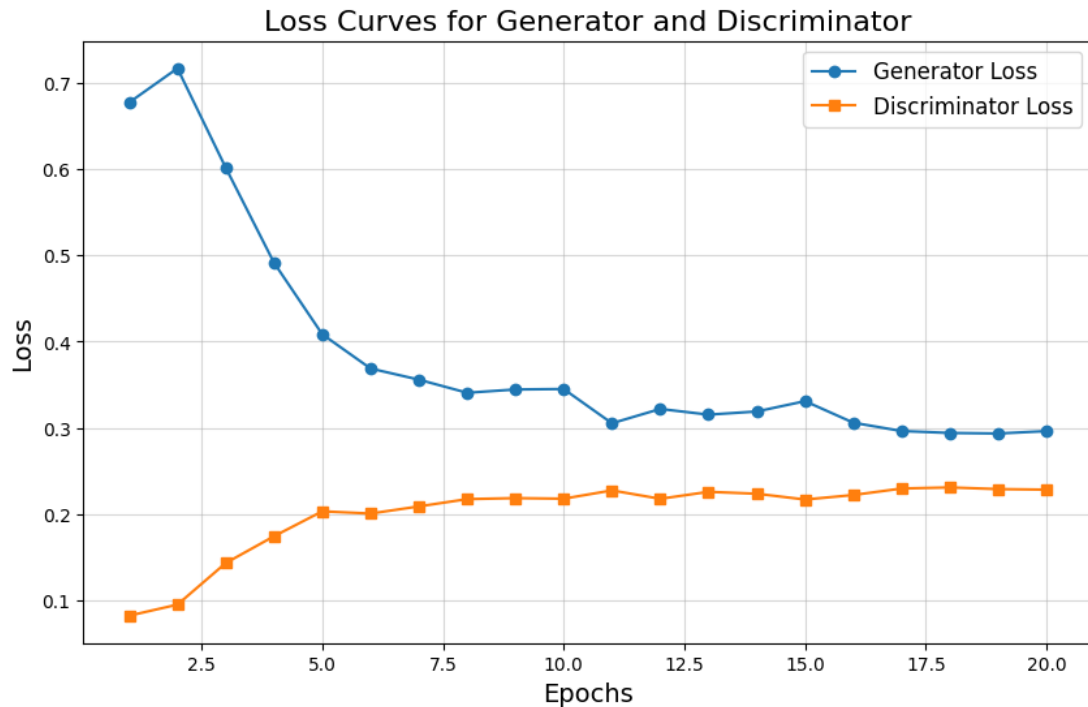
Generator Loss:

The performance starts high (approximately 1.6) at the beginning of training and drops sharply within the first few epochs, stabilizing around 0.8 to 1.0. After stabilization, it exhibits minor fluctuations, indicating that the generator continues to improve incrementally but struggles to achieve significant progress.

Discriminator Loss:

The loss begins at a low value of approximately 0.4, indicating that the discriminator is initially effective at distinguishing real images from fake ones. It rises sharply during the first few epochs, peaking near 0.6, before stabilizing and fluctuating around that value.

Loss Curves of LSGAN Model



(2) Description of the LSGAN Loss Curves

Generator Loss:

The process begins at a relatively high value of approximately 0.7 but decreases significantly during the initial few epochs, eventually stabilizing at lower values between 0.2 and 0.3. The overall trend is smoother, exhibiting minimal fluctuations, which indicates stable training dynamics.

Discriminator Loss:

The performance starts very low (approximately 0.1) and increases steadily, stabilizing at around 0.2 to 0.3 after a few epochs. The discriminator maintains consistent effectiveness without significant fluctuations.

(3) Explaining the Shapes of DCGAN Loss Curves

Generator Loss:

Initially, the loss is high because the generator produces poor-quality images that the discriminator can easily identify as fake. As the generator improves and learns to create more realistic images, the loss decreases. However, the slower stabilization indicates the challenges faced by DCGAN in maintaining a stable balance between the generator and discriminator. Fluctuations in later epochs suggest some instability in adversarial training, where the generator and discriminator may over compete.

Discriminator Loss:

Initially, the loss is low because the discriminator can easily distinguish real images from fake ones. It increases as the generator improves and begins to create more realistic images, making it more challenging for the discriminator to classify them accurately. Stabilization occurs when the discriminator becomes more adept at adapting to the generator's outputs, indicating a relative balance between the two networks.

(4) Explaining the Shapes of LSGAN Loss Curves

Generator Loss:

The consistent decrease demonstrates the generator's capacity for continuous improvement, leveraging the least-squares loss function, which penalizes outputs based on their deviation from the target rather than relying on binary classification. Smoother trends indicate enhanced training stability, exhibiting less oscillation compared to DCGAN.

Discriminator Loss:

Low initial values indicate the discriminator's initial strength, as the generator begins with low-quality outputs. A gradual increase and stabilization suggest that the discriminator effectively balances its learning to detect fake images while preventing it from overpowering the generator. This balance is a result of the enhanced training dynamics in LSGAN.

(5) Relation of Loss Values to Image Quality

DCGAN Model:

Higher generator loss during the early epochs corresponds to low-quality, noisy images (e.g., Epoch 0). As the loss decreases, the images improve in clarity and realism (e.g., Epoch 29). However, residual oscillations in the loss curves are associated with occasional distortions in the generated digits. The stabilization of discriminator loss indicates a balance in which neither network dominates; however, minor instability could still result in artifacts in the images.

LSGAN Model:

Lower and more stable generator loss corresponds to consistent improvements in image quality over the epochs. By Epoch 19, the generated digits are sharper and more realistic, reflecting the benefits of least-squares loss in enhancing stability and convergence. The stable discriminator loss further supports balanced adversarial training, ensuring that both networks effectively contribute to the enhancement of image quality.

- 3. Show your best model based on FID ↓ metric. Describe the structure, hyper parameters, loss function, etc., that you modified to achieve lower FID. If your model gives 120 or higher FID, give some possible rationales for such result.**

(1) Model Structure for Both DCGAN and LSGAN

Generator:

The generator is responsible for producing synthetic data, such as images, from random latent vectors. Its architecture comprises a series of transposed convolutions and standard convolutions, interspersed with batch normalization and activation functions. This design aims to upsample the latent vector into an image. The latent dimension represents the size of the input noise vector.

Layer 1:

The input latent vector is upsampled into a small feature map with dimensions of $256 \times 2 \times 2$ using a transposed convolution, which incorporates batch normalization and a ReLU activation function. This convolutional layer enhances feature extraction.

Layer 2:

The feature map is further upsampled to a size of $256 \times 6 \times 6$ using two transposed convolutions.

Layer 3:

The process continues with upsampling to a resolution of $128 \times 14 \times 14$ through multiple transposed convolutions, followed by a convolution to refine the features.

Layer 4:

Final upsampling to the target image size of $1 \times 28 \times 28$ (grayscale image) is achieved using transposed convolutions, followed by refinement through standard convolutions. A Tanh activation function is applied at the end to map pixel values to the range of $[-1, 1]$.

Forward Pass:

The input is processed sequentially through Layer 1, Layer 2, Layer 3, and Layer 4, resulting in a grayscale image with dimensions of $1 \times 28 \times 28$ pixels.

```
Generator(  
  (layer1): Sequential(  
    (0): TConv_BN_ReLU(  
      (0): ConvTranspose2d(60, 256, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)  
      (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      (2): ReLU(inplace=True)  
    )  
    (1): Conv_BN_ReLU(  
      (0): Conv2d(256, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
      (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      (2): ReLU(inplace=True)  
    )  
  )  
  (layer2): Sequential(  
    (0): TConv_BN_ReLU(  
      (0): ConvTranspose2d(128, 256, kernel_size=(4, 4), stride=(1, 1), padding=(1, 1), bias=False)  
      (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      (2): ReLU(inplace=True)  
    )  
    (1): TConv_BN_ReLU(  
      (0): ConvTranspose2d(256, 256, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)  
      (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      (2): ReLU(inplace=True)  
    )  
  )  
)
```

```

(layer3): Sequential(
  (0): TConv_BN_ReLU(
    (0): ConvTranspose2d(256, 128, kernel_size=(4, 4), stride=(1, 1), padding=(1, 1), bias=False)
    (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU(inplace=True)
  )
  (1): TConv_BN_ReLU(
    (0): ConvTranspose2d(128, 128, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU(inplace=True)
  )
  (2): Conv_BN_ReLU(
    (0): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU(inplace=True)
  )
)
(layer4): Sequential(
  (0): TConv_BN_ReLU(
    (0): ConvTranspose2d(128, 64, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU(inplace=True)
  )
  (1): Conv_BN_ReLU(
    (0): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU(inplace=True)
  )
  (2): Conv_BN_ReLU(
    (0): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU(inplace=True)
  )
  (3): Conv2d(64, 1, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (4): Tanh()
)
)

```

Discriminator:

The discriminator functions as a classifier that differentiates between real images and those generated by the Generator. It employs convolutional layers to downsample the input image and extract features, followed by a fully connected layer that predicts a single output value: either real or fake. A grayscale image with dimensions of 1 x 28 x 28 serves as the input.

Convolutional Layers:

A series of convolutional blocks progressively downsampled the image to a feature vector of size 64 x 1 x 1. Each block consists of a convolutional layer, followed by batch normalization and a Leaky ReLU activation function.

Fully Connected Layer:

The flattened feature vector is transformed into a single output value through a linear layer.

Forward Pass:

The input is processed through the convolutional layers. A scalar value that indicates whether the output is real (high value) or fake (low value). Intermediate feature representations are extracted before the fully connected layer, which can be useful for interpretability or auxiliary tasks.

```
Discriminator(  
  (conv): Sequential(  
    (0): Conv_BN_LReLU(  
      (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)  
      (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      (2): LeakyReLU(negative_slope=0.01, inplace=True)  
    )  
    (1): Conv_BN_LReLU(  
      (0): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
      (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      (2): LeakyReLU(negative_slope=0.01, inplace=True)  
    )  
    (2): Conv_BN_LReLU(  
      (0): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)  
      (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      (2): LeakyReLU(negative_slope=0.01, inplace=True)  
    )  
    (3): Conv_BN_LReLU(  
      (0): Conv2d(128, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)  
      (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      (2): LeakyReLU(negative_slope=0.01, inplace=True)  
    )  
    (4): Conv_BN_LReLU(  
      (0): Conv2d(128, 64, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)  
      (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      (2): LeakyReLU(negative_slope=0.01, inplace=True)  
    )  
  )  
  (fc): Linear(in_features=64, out_features=1, bias=True)  
)
```

(2) Hyperparameter Settings

Model Type	Batch Size	Learning Rate	Epochs	Latent Dimension
DCGAN	200	0.0002	30	100
LSGAN	200	0.0002	20	60

(3) Loss Functions for DCGAN and LSGAN

DCGAN	Binary Cross-Entropy with Logits Loss
LSGAN	Mean Squared Error Loss

Binary Cross-Entropy with Logits Loss (BCE):

This loss function, specifically designed for binary classification tasks, integrates a sigmoid activation function with binary cross-entropy loss to quantify the disparity between the discriminator's predicted probability (real or fake) and the target label. In DCGAN, the discriminator aims to minimize this loss to accurately classify real and fake samples, while the generator seeks to minimize it in order to the discriminator into classifying its fake samples as real.

Mean Squared Error Loss (MSE):

This loss function employs mean squared error to quantify the difference between the predicted values and the targets, where the target values for real samples are typically set to 1.0 and those for fake samples are set to 0.0. In LSGAN, the least-squares loss replaces the sigmoid cross-entropy loss, enhancing gradient stability during training. The discriminator minimizes the squared error for both real and fake samples, while the generator minimizes the squared error between its fake samples and the target values for real samples.

Key Differences Between BCE and MSE:

Including their gradient behavior and impact on training stability. With BCE, the sigmoid activation function can saturate, leading to vanishing gradients when the discriminator becomes overly confident. This saturation may result in training instability in DCGAN. In contrast, MSE provides more stable gradients, even when the discriminator is highly confident, thereby reducing the risk of vanishing gradients. This makes LSGAN more stable during training, as their smoother loss landscape facilitates the generator's ability to learn better representations.

(4) Optimizer and Scheduler for Both Generator and Discriminator

Optimizer	Beta1	Beta2	Scheduler	Step Size	gamma
Adam	0.5	0.999	StepLR	5	0.5

Optimizer:

The Adam optimizer is well-suited for deep learning tasks because of its adaptive learning rate and momentum-based updates. The learning rate is set to 0.0002, and the hyperparameters, referred to as betas, are chosen to balance the speed and stability of convergence. These betas control the exponential decay rates of the moving averages for the past gradient (0.5) and the squared gradient (0.999).

Scheduler:

The StepLR scheduler decreases the learning rate by multiplying it by a factor of gamma (0.5) after every 5 epochs. This gradual reduction in the learning rate helps models converge more smoothly by allowing finer updates to the weights in the later stages of training, thereby reducing the risk of overshooting the optimal solution.

(5) Fréchet Inception Distance (FID)

The Fréchet Inception Distance (FID) is a widely used metric for evaluating the quality of images generated by generative models like GANs. It measures the similarity between the distribution of generated images and that of real images. A lower FID score signifies higher quality and greater diversity in the generated images, as it indicates that the distributions of real and generated images are more closely aligned.

Model Type	FID
DCGAN	112.98
LSGAN	103.03

References

<https://chih-sheng->

[huang821.medium.com/pytorch%E6%89%8B%E6%8A%8A%E6%89%8B%E5%AF%A6%E4%BD%9C-generative-adversarial-network-8adae9a3092b](https://chih-sheng-huang821.medium.com/pytorch%E6%89%8B%E6%8A%8A%E6%89%8B%E5%AF%A6%E4%BD%9C-generative-adversarial-network-8adae9a3092b)

<https://ithelp.ithome.com.tw/articles/10303526>

<https://github.com/Lornatang/DCGAN-PyTorch>

<https://medium.com/@manoharmanok/implementing-dcgan-in-pytorch-using-the-celeba-dataset-a-comprehensive-guide-660e6e8e29d2>